

Syed Sualeh abbas 10303

Syed Mohsin Mujtaba Rizvi mm10016

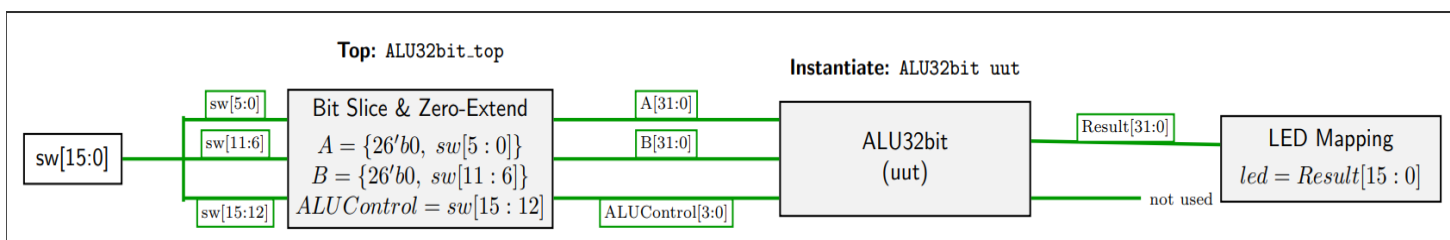
Syed zain naqvi

Lab 7: Design and FPGA Implementation of the Register File for a Single-Cycle RISC-V Processor

Objective

Task 1: Register File Block Diagram Design

Block diagram of Register File





Task 2: Verilog Description and Testbench Development

Verilog source file for Register File

```
10 // x0 always reads 0, writes ignored
11 ///////////////////////////////////////////////////////////////////
12
13 module RegisterFile(
14     input clk,                // Clock
15     input rst,                // Synchronous reset
16     input writeEnable,        // Write enable
17     input [4:0] readAddr1,    // Read port 1 address
18     input [4:0] readAddr2,    // Read port 2 address
19     input [4:0] writeAddr,     // Write address
20     input [31:0] writeData,    // Data to write
21     output [31:0] readData1,   // Data read port 1
22     output [31:0] readData2   // Data read port 2
23 );
24
25 // 32 registers, each 32 bits
26 reg [31:0] regs [31:0];
27 integer i;
28
29 // -----
30 // Read ports (combinational)
31 // -----
32 // x0 always returns 0
33 assign readData1 = (readAddr1 == 5'b00000) ? 32'b0 : regs[readAddr1];
34 assign readData2 = (readAddr2 == 5'b00000) ? 32'b0 : regs[readAddr2];
35
36 // -----
37 // Write port (synchronous)
38 // -----
39 always @(posedge clk) begin
40     if (rst) begin
41         // Clear all registers on reset
42         for (i = 0; i < 32; i = i + 1)
43             regs[i] <= 32'b0;
44     end else if (writeEnable && writeAddr != 5'b00000) begin
45         // Write data to register if write enabled and not x0
46         regs[writeAddr] <= writeData;
47     end
48 end
49
50 endmodule
51
```



TEST BENCH CODE:

```
`timescale 1ns / 1ps
/////////////////////////////////////////////////////////////////
// Company:
// Engineer:
//
// Design Name: RF + ALU + FSM Testbench
// Module Name: RF_ALU_FSM_tb
// Description: Thorough testbench that writes multiple registers,
//              performs multiple ALU operations, prints all signals
/////////////////////////////////////////////////////////////////
```

```
module RF_ALU_FSM_tb;
```

```
    // Clock and reset
    reg clk;
    reg rst;
```

```
    // Register File signals
    reg writeEnable;
    reg [4:0] rs1, rs2, rd; // read/write addresses
    reg [31:0] writeData;
    wire [31:0] readData1, readData2;
```

```
    // ALU signals
    reg [3:0] ALUControl;
    wire [31:0] ALUResult;
    wire Zero;
    reg [31:0] A, B; // ALU inputs
```

```
    // FSM state definition
    reg [3:0] state;
    localparam IDLE = 0,
        WRITE_REGS = 1,
        ALU_OP = 2,
        WRITE_ALU_RES = 3,
        NEXT_ALU_OP = 4,
        DONE = 5;
```

```
    integer i;
```

```
    // Instantiate Register File
    RegisterFile RF(
        .clk(clk),
        .rst(rst),
        .writeEnable(writeEnable),
        .readAddr1(rs1),
        .readAddr2(rs2),
        .writeAddr(rd),
        .writeData(writeData),
        .readData1(readData1),
        .readData2(readData2)
    );
```

```
    // Instantiate ALU
    ALU32bit ALU(
        .A(A),
        .B(B),
        .ALUControl(ALUControl),
        .ALUResult(ALUResult),
        .Zero(Zero)
    );
```

```
    // Clock generation: 10ns period
    initial clk = 0;
    always #5 clk = ~clk;
```

```
    // Arrays of test data
    reg [31:0] regValues [1:10]; // Values to write
    reg [4:0] regDest [1:10]; // Destination registers
    reg [3:0] aluOps [0:7]; // ALU operations
```

```
    initial begin
```

```
        // Example register values
        regValues[1] = 32'h10101010;
        regValues[2] = 32'h01010101;
        regValues[3] = 32'h00000000;
        regValues[4] = 32'hFFFFFFF;
        regValues[5] = 32'h0000FFFF;
        regValues[6] = 32'hAAAA5555;
        regValues[7] = 32'h12345678;
        regValues[8] = 32'h87654321;
        regValues[9] = 32'h0F0F0F0F;
        regValues[10] = 32'hF0F0F0F0;
```

```
        // Destination registers to write ALU results
```

```
        regDest[1] = 4;
        regDest[2] = 5;
        regDest[3] = 6;
        regDest[4] = 7;
        regDest[5] = 8;
        regDest[6] = 9;
        regDest[7] = 10;
        regDest[8] = 11;
        regDest[9] = 12;
        regDest[10] = 13;
```

```
    // ALU operations
    aluOps[0] = 4'b0010; // ADD
    aluOps[1] = 4'b0110; // SUB
    aluOps[2] = 4'b0000; // AND
    aluOps[3] = 4'b0001; // OR
    aluOps[4] = 4'b0011; // XOR
    aluOps[5] = 4'b0100; // SLL
    aluOps[6] = 4'b0101; // SRL
    aluOps[7] = 4'b0111; // SLT
end
```

```
    // FSM sequence
    reg [3:0] aluIndex;
    reg [3:0] writeIndex;
    always @(posedge clk or posedge rst) begin
```

```
        if (rst) begin
            state <= IDLE;
            writeEnable <= 0;
            rs1 <= 0; rs2 <= 0; rd <= 0;
            writeData <= 0;
            ALUControl <= 0;
            writeIndex <= 1;
            aluIndex <= 0;
        end else begin
            case(state)
                IDLE: state <= WRITE_REGS;
```

```
            // Write multiple registers
            WRITE_REGS: begin
                if (writeIndex <= 10) begin
                    writeEnable <= 1;
                    rd <= writeIndex;
                    writeData <= regValues[writeIndex];
                    writeIndex <= writeIndex + 1;
                end else begin
                    writeEnable <= 0;
                    writeIndex <= 1;
                    state <= ALU_OP;
                end
            end
        end
```

```
    // ALU operations between x1 and x2 repeatedly
```

```
    ALU_OP: begin
        rs1 <= 1; rs2 <= 2;
        A <= readData1; B <= readData2;
        ALUControl <= aluOps[aluIndex];
        state <= WRITE_ALU_RES;
    end
```

```
    WRITE_ALU_RES: begin
        writeEnable <= 1;
        rd <= regDest[aluIndex+1];
        writeData <= ALUResult;
        state <= NEXT_ALU_OP;
    end
```

```
    NEXT_ALU_OP: begin
        writeEnable <= 0;
        if (aluIndex < 7) begin
            aluIndex <= aluIndex + 1;
            state <= ALU_OP;
        end else begin
            state <= DONE;
        end
    end
```

```
    DONE: begin
        $display("Test Completed!");
        $finish;
    end
```

```
endcase
end
end
```

```
    // Monitor all outputs
```

```
    initial begin
```

```
        $monitor("Time=%0t | State=%0d | rs1=%0d rs2=%0d rd=%0d | writeEn=%b | readData1=%h readData2=%h | A=%h B=%h | ALUResult=%h Zero=%b",
```

```
            $time, state, rs1, rs2, rd, writeEnable, readData1, readData2, A, B, ALUResult, Zero);
```

```
    end
```

```
    // Waveform dump for GTKWave or ModelSim
```

```
    initial begin
```

```
        $dumpfile("RF_ALU_FSM_tb.vcd");
        $dumpvars(0, RF_ALU_FSM_tb);
```

```
    end
```

```
    // Initial reset
```

```
    initial begin
        rst = 1;
        #10 rst = 0;
    end
```

```
endmodule
```



Verilog source file for Top-level (top rf alu.v).

```
`timescale 1ns / 1ps

module TOPmodule_registers(

    input clk,
    input rst,
    input [15:0] switches,
    output [15:0] leds

);

// =====
// WIRES
// =====
wire [31:0] readData1;
wire [31:0] readData2;
wire [31:0] ALUResult;
wire Zero;

reg writeEnable;
reg [4:0] writeAddr;

wire [31:0] writeData;

// =====
// CONSTANTS (for quick verification)
// =====
parameter CONST_A = 32'h10101010;
parameter CONST_B = 32'h01010101;

// =====
// WRITE ADDRESS CONTROL
// =====
always @(*) begin
    case(switches[6:5])

        2'b01: writeAddr = 5'd1; // load x1
        2'b10: writeAddr = 5'd2; // load x2
        2'b11: writeAddr = 5'd3; // store ALU result

        default: writeAddr = 5'd0;

    endcase
end

// =====
// WRITE ENABLE
// =====
always @(*) begin
    writeEnable = switches[0];
end

// =====

// WRITE DATA MUX
// =====
assign writeData =
    (switches[6:5] == 2'b01) ? CONST_A :
    (switches[6:5] == 2'b10) ? CONST_B :
    (switches[6:5] == 2'b11) ? ALUResult :
    32'b0;

// =====
// REGISTER FILE
// =====
RegisterFile RF (

    .clk(clk),
    .rst(rst),

    .writeEnable(writeEnable),

    .readAddr1(5'd1), // always read x1
    .readAddr2(5'd2), // always read x2

    .writeAddr(writeAddr),
    .writeData(writeData),

    .readData1(readData1),
    .readData2(readData2)
);

// =====
// ALU
// =====
ALU32bit ALU (

    .A(readData1),
    .B(readData2),

    .ALUControl(switches[4:1]),

    .ALUResult(ALUResult),
    .Zero(Zero)
);

// =====
// LED OUTPUT
// =====
assign leds[15] = Zero; // show Zero flag
assign leds[14:0] = ALUResult[14:0]; // show
result LSBs

endmodule
```



Testbench file for Top-level

```
`timescale 1ns / 1ps

module TOPmodule_registers_tb;

    reg clk;
    reg rst;
    reg [15:0] switches;
    wire [15:0] leds;

    // Instantiate DUT
    TOPmodule_registers DUT (
        .clk(clk),
        .rst(rst),
        .switches(switches),
        .leds(leds)
    );

    // Clock generation (10ns period)
    always #5 clk = ~clk;

    // =====
    // TASK: Perform a write
    // =====
    task write_operation;
        input [1:0] addrSel;
        begin
            switches[6:5] = addrSel; // select x1, x2,
or x3
            switches[0] = 1; // write enable
            @(posedge clk);
            #2 switches[0] = 0; // disable write
        end
    endtask

    // =====
    // TASK: Perform ALU operation
    // =====
    task alu_operation;
        input [3:0] aluCtrl;
        begin
            switches[4:1] = aluCtrl;
            @(posedge clk);
            #2;
            $display("TIME=%0t | ALUControl=%b |
LEDS=%h | ZERO=%b",
                $time, aluCtrl, leds, leds[15]);
        end
    endtask

    // =====
    // TEST SEQUENCE

// =====
initial begin

    $display("===== STARTING
SIMULATION =====");

    clk = 0;
    rst = 1;
    switches = 0;

    #20;
    rst = 0;

    // -----
    // Load CONST_A into x1
    // -----
    $display("Loading CONST_A into x1");
    write_operation(2'b01);

    // -----
    // Load CONST_B into x2
    // -----
    $display("Loading CONST_B into x2");
    write_operation(2'b10);

    // -----
    // ALU OPERATIONS
    // -----
    alu_operation(4'b0010); // ADD
    alu_operation(4'b0110); // SUB
    alu_operation(4'b0000); // AND
    alu_operation(4'b0001); // OR
    alu_operation(4'b0011); // XOR
    alu_operation(4'b0111); // SLT

    // -----
    // Store ALU result into x3
    // -----
    $display("Storing ALU result into x3");
    write_operation(2'b11);

    #50;
    $display("===== SIMULATION
FINISHED =====");
    $stop;

end

endmodule
```



Simulation waveforms showing correct operation

ALU OPERATION BITS:

```
// ALU operations
aluOps[0] = 4'b0010; // ADD
aluOps[1] = 4'b0110; // SUB
aluOps[2] = 4'b0000; // AND
aluOps[3] = 4'b0001; // OR
aluOps[4] = 4'b0011; // XOR
aluOps[5] = 4'b0100; // SLL
aluOps[6] = 4'b0101; // SRL
aluOps[7] = 4'b0111; // SLT
```

Expected Results:

Register	Written Value
x4	ADD result 0x11111111
x5	SUB result 0x0F0F0F0F
x6	AND result 0x00000000
x7	OR result 0x11111111
x8	XOR result 0x11111111
x9	SLL result 0x20202020
x10	SRL result 0x08080808
x11	SLT result 0x00000000

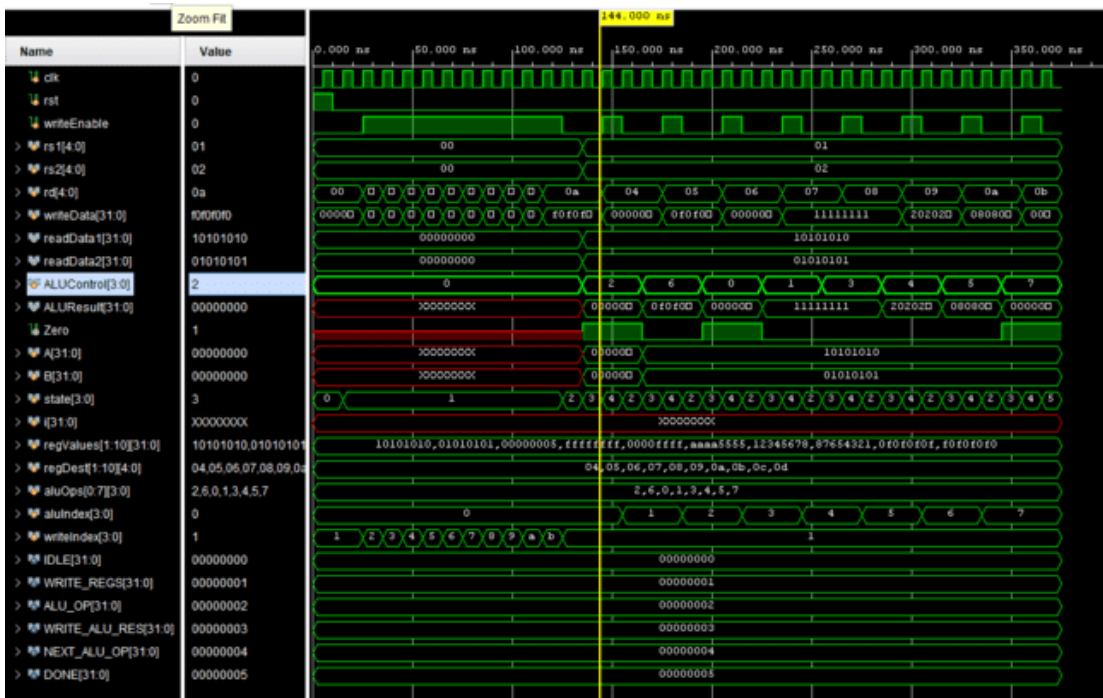


WAVEFORMS:

When NO VALUES ARE ADDED TO REGISTERS FOR ALU INPUTS: (A, B; Both has value of zero)



AFTER WRITE DATA IS TURNED ON THE HARD CODED VALUES GET SAVED IN READ DATA1, READ DATA2





WHEN ALU CONTROL=0, THAT MEANS IT SHOULD PERFORM AND:



When ALU CONTROL=1, THAT MEANS IT SHOULD PERFORM OR:

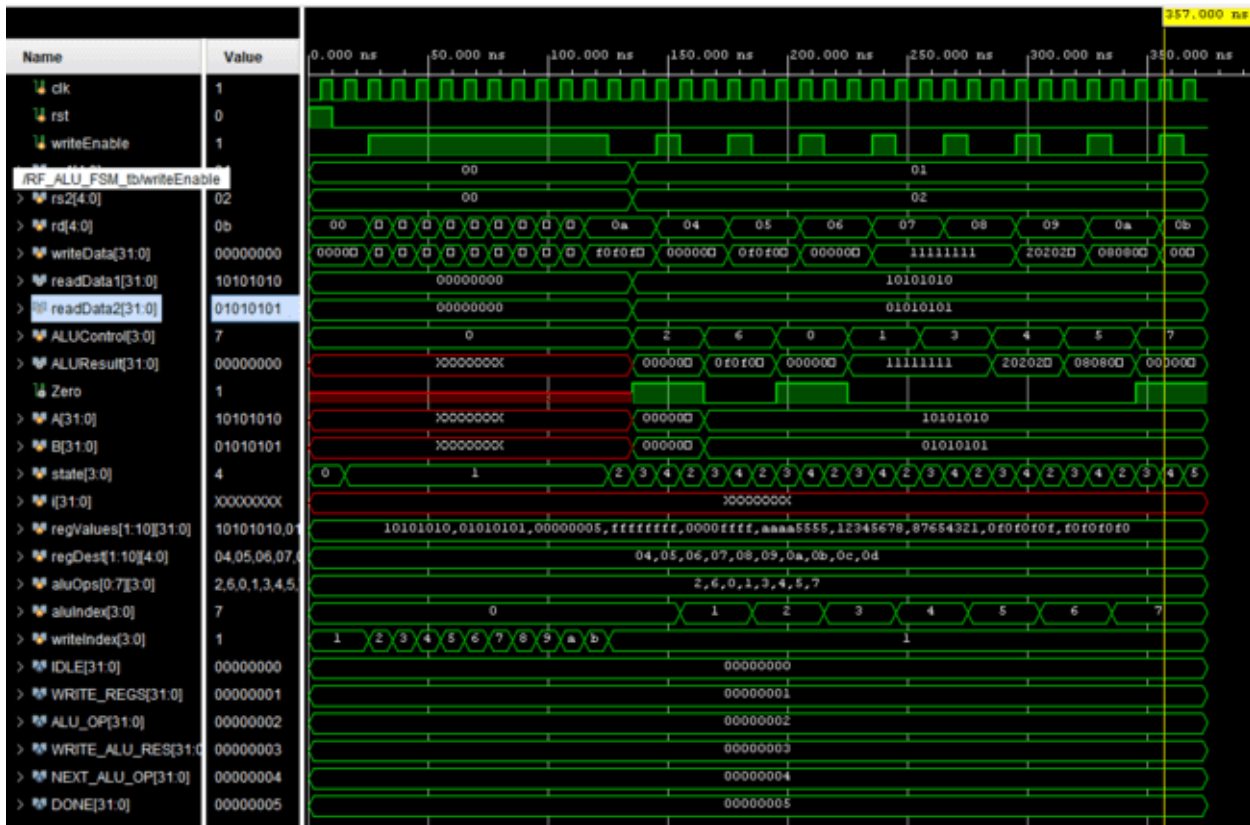


WHEN ALU CONTROL=5, THAT MEANS IT SHOULD PERFORM SRL:





WHEN ALU CONTROL=7, THAT MEANS IT SHOULD PERFORM SLT:

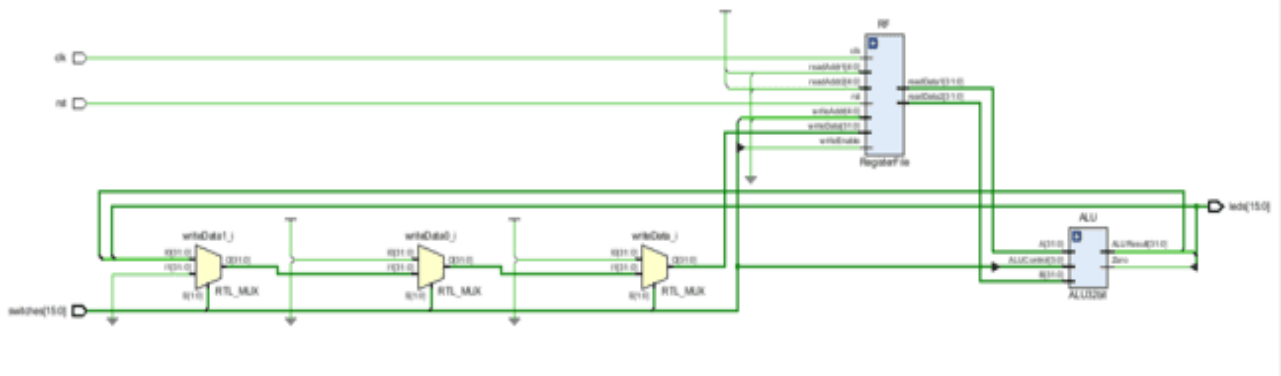


REST CASES ARE ALSO DONE SAME WAY.....

Task 3: FPGA Synthesis and RTL Analysis

Note: The switch interface, button debouncing module, and LED interface must be re used from Lab 5.

RTL schematic screenshots (register file array, decoder, FSM, ALU connections)



Demonstration of FPGA operation

VALUE STORD IN A= 0X10101010

B=0X01010101

◆ AND → 0000

Result

Code

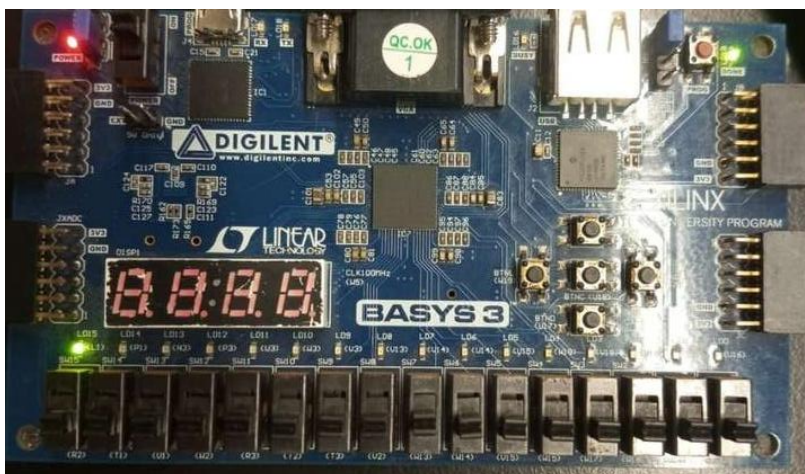
0x00000000

LEDs

Code

LED15 = 1
All others = 0

ON LEDs

 none

◆ OR → 0001

Result

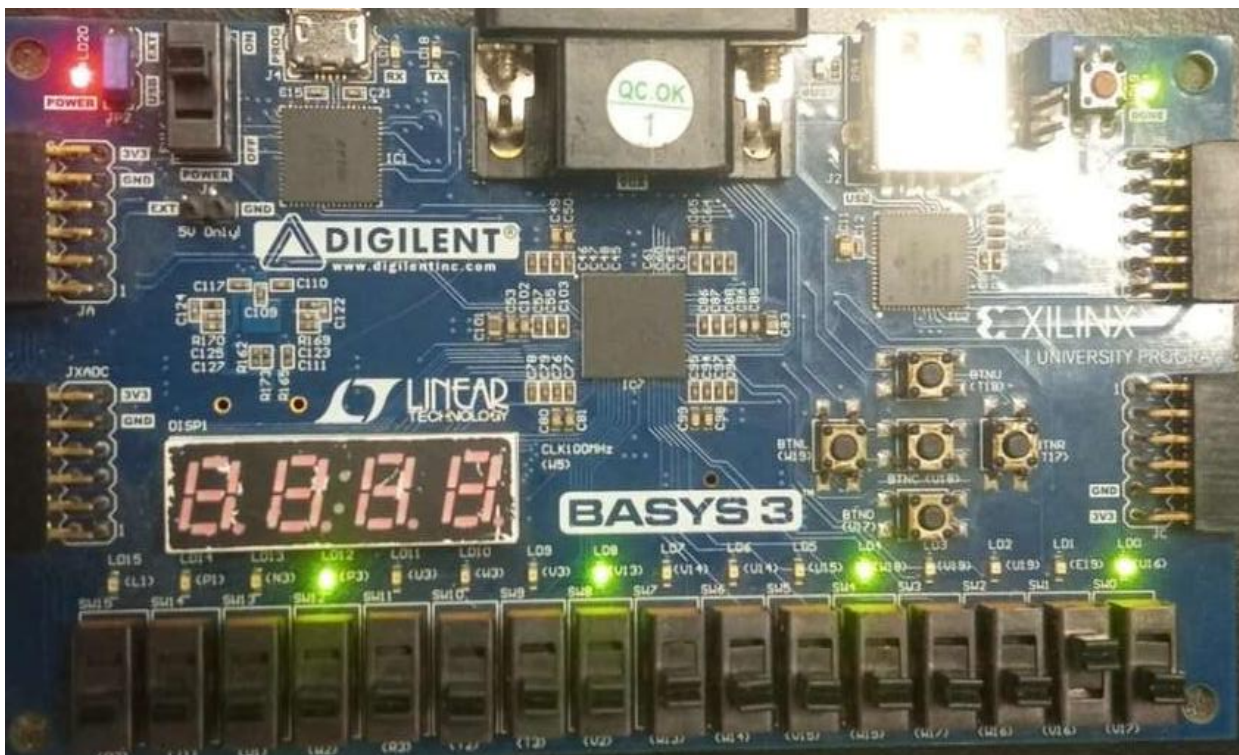
Code

```
0x11111111
lower 16 bits → 0x1111 → 0001 0001 0001 0001
```

ON LEDs

→ LED0, LED4, LED8, LED12

Zero flag OFF.



◆ ADD → 0010

Result

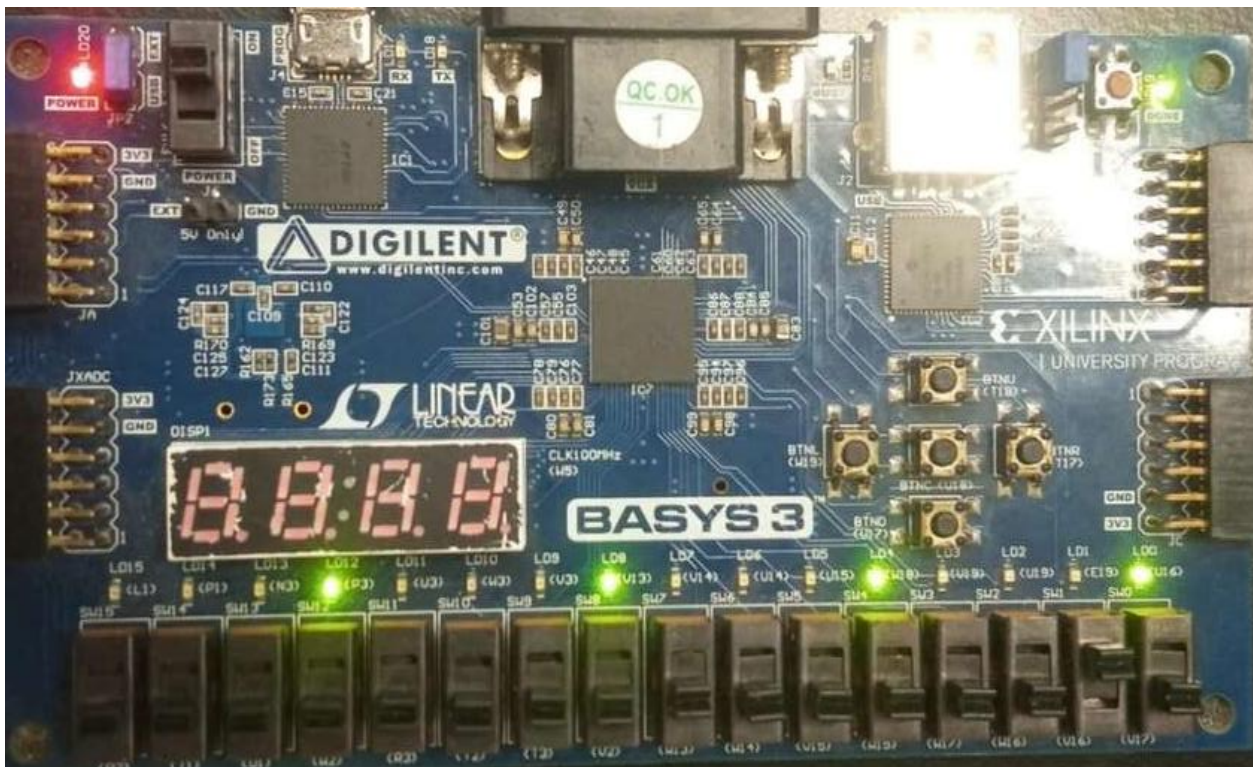
Code

```
0x10101010 + 0x01010101 = 0x11111111
```

ON LEDs

→ LED0, LED4, LED8, LED12

Zero flag OFF.



◆ XOR → 0011

Result

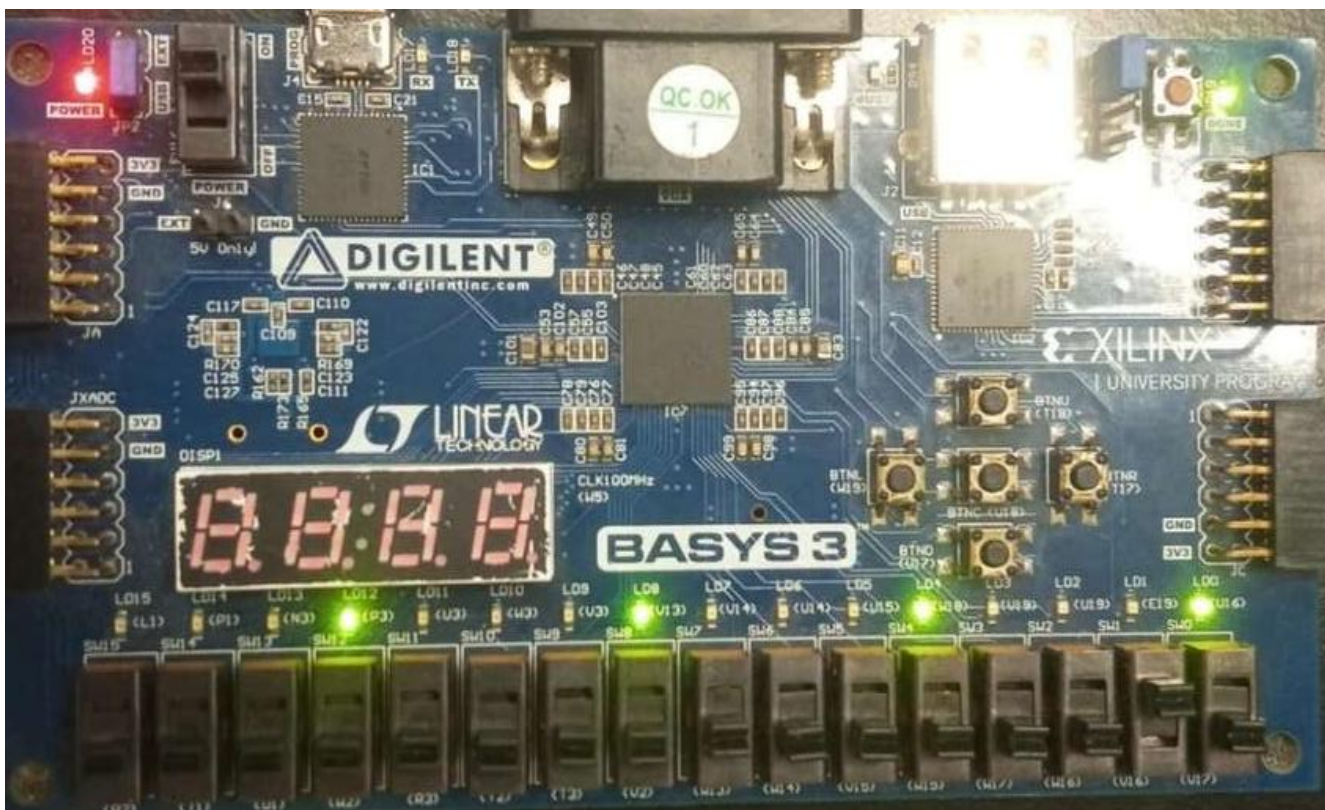
Code

0x11111111

ON LEDs

→ LED0, LED4, LED8, LED12

Zero flag OFF.



◆ SLL → 0100

Shift amount = lower 5 bits of B

Code

B = 0x01010101 → shift by 1

Result

Code

0x20202020

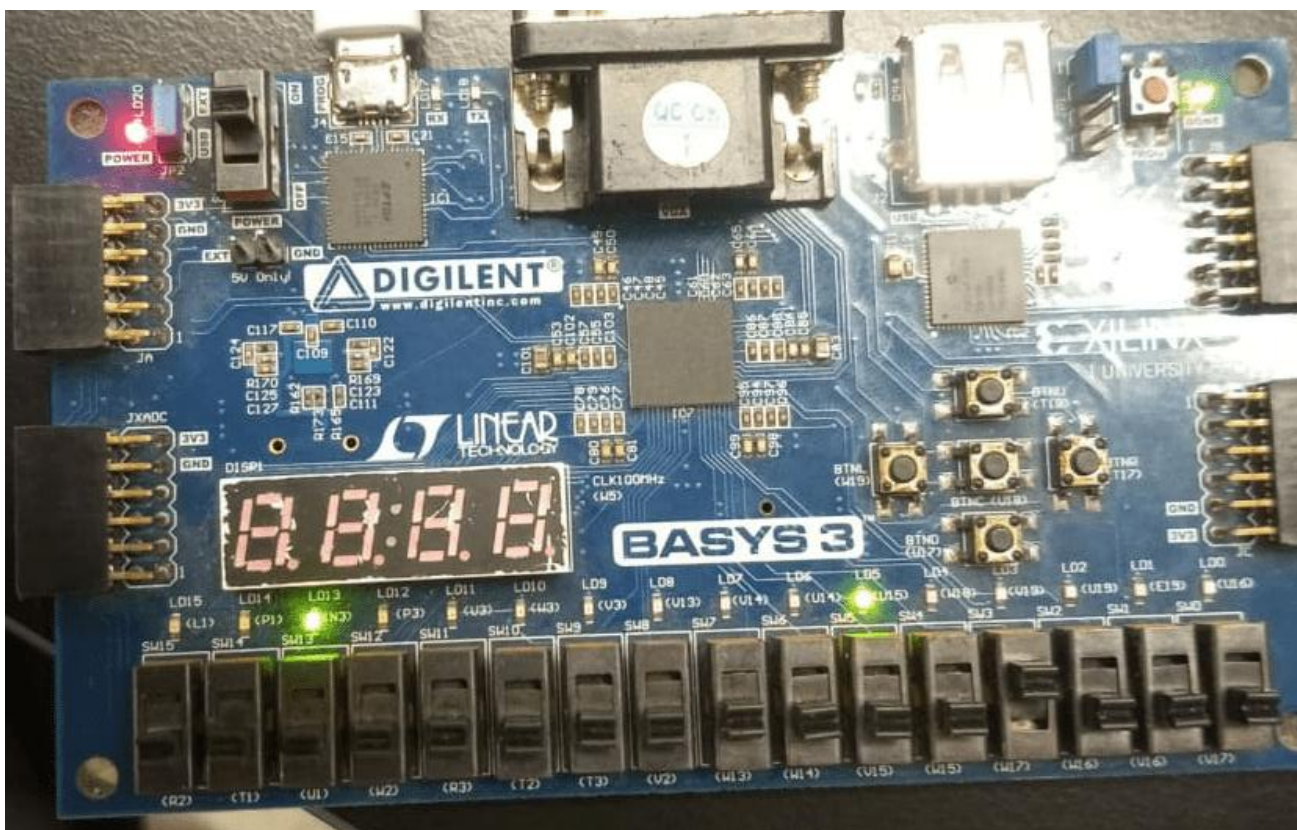
lower 16 bits → 0x2020

binary → 0010 0000 0010 0000

ON LEDs

→ LED5, LED13

Zero flag OFF.



◆ SRL → 0101

Result

0x08080808

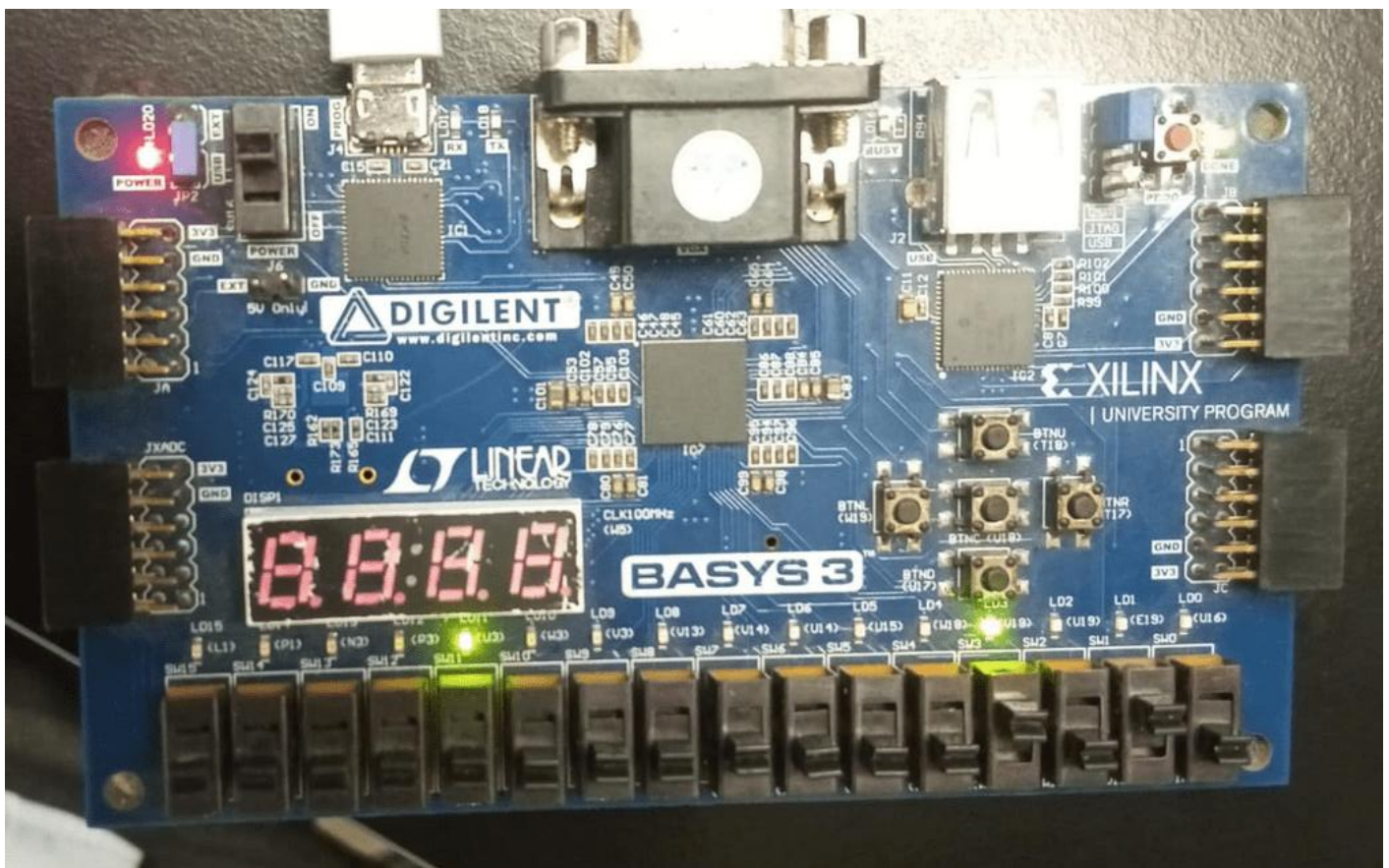
lower 16 bits → 0x0808

binary → 0000 1000 0000 1000

ON LEDs

→ LED3, LED11

Zero flag OFF.



◆ SUB → 0110

Result

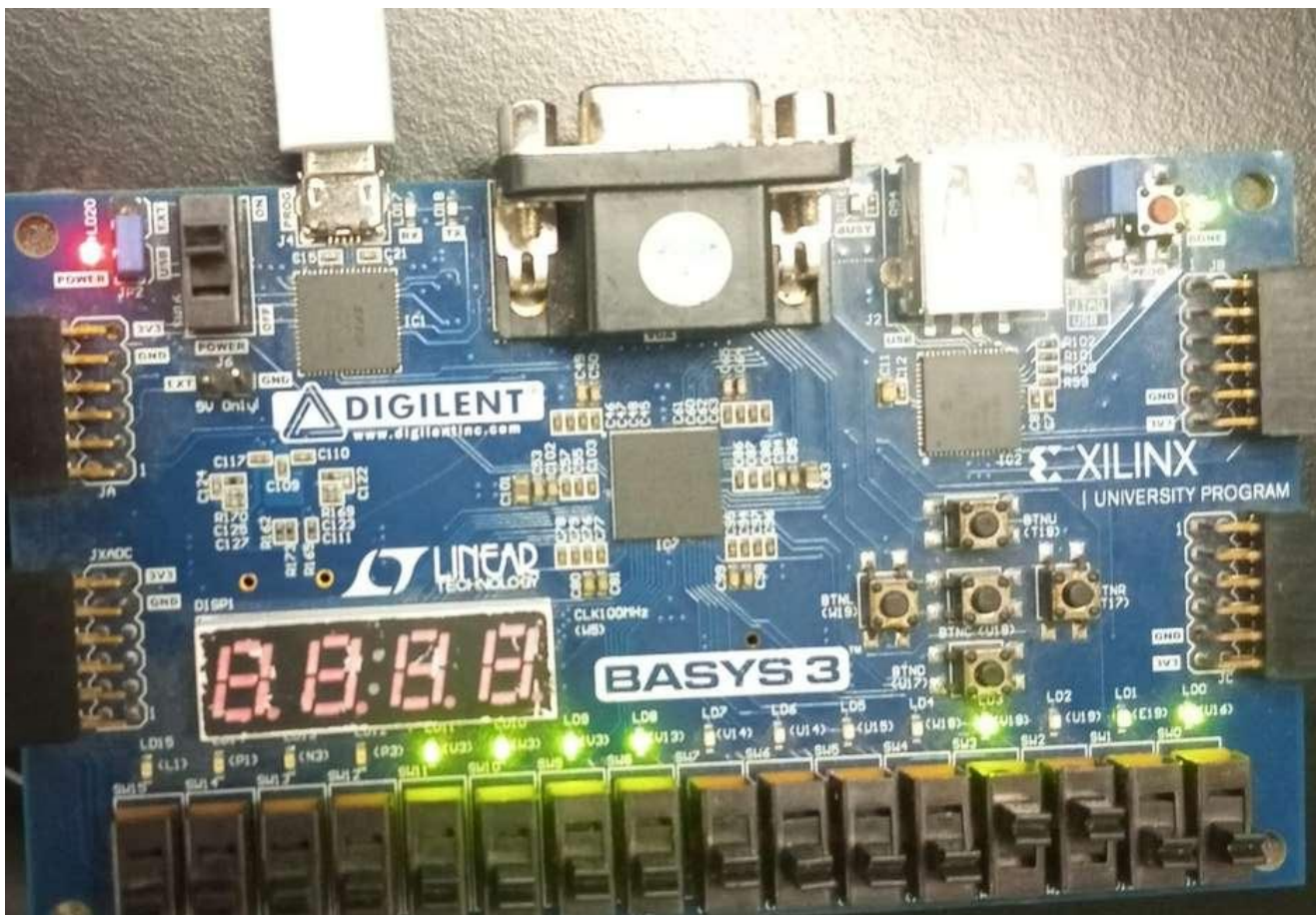
</> Code

```
0x10101010 - 0x01010101 = 0x0F0F0F0F
lower 16 bits → 0x0F0F
binary → 0000 1111 0000 1111
```

ON LEDs

→ LED0, LED1, LED2, LED3, LED8, LED9, LED10, LED11

Zero flag OFF.



NOTE: THE 2 LED'S WHICH ARE NOT WORKING LED: 1, 2; WERE LIGHTLY LIT BT WERE NO CAPTURED IN THE PICTURE.

◆ SLT → 0111

$A > B \rightarrow \text{false}$

Result

Code

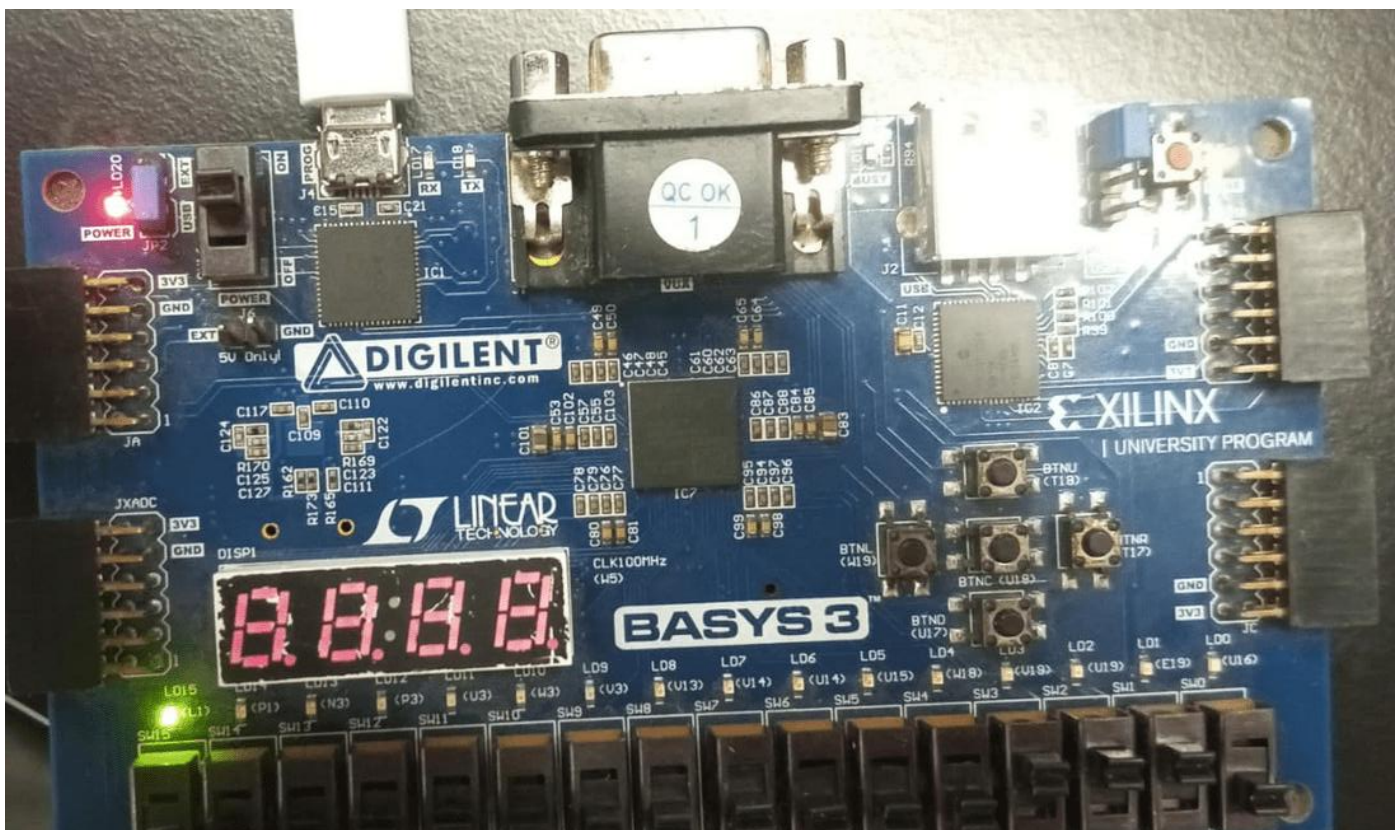
0x00000000

LEDs

Code

LED15 = 1

All others = 0





Assessment Rubric

Lab 7: Design and FPGA Implementation of the Register File for a Single-Cycle RISC-V Processor

Name:	Student ID:	Section:
-------	-------------	----------

Points Distribution

	Task No.	LR 2 (Code)	LR 5 (Results)	LR 11 (Design)
In-Lab	Task 1	-	-	/10
	Task 2	/20	/10	-
	Task 3	/20	/20	-
Total Points		/40	/30	/10
CLO Mapped		CLO 1		

Affective Domain Rubric		Points	CLO Mapped
AR7	Report Submission & Git Upload	/10 & /10	CLO 1

CLO	Total Points	Points Obtained
1	100	
Total	100	

For description of different levels of the mapped rubrics, please refer to the Lab Evaluation Assessment Rubrics and Affective Domain Assessment Rubrics provided here.